Intelligent Concurrent Proxy Gateway: Project Guide

This document explains the project in everyday language. It describes what the gateway is for, how one request travels through it, and what each important program and code module does.

1. The idea in one sentence

The gateway is a traffic manager between client devices and a list of proxy servers: it receives a web request, picks a suitable proxy, sends the request through that proxy, and returns the result to the client.

```
Your app or device  ->  This gateway  ->  One selected proxy  ->  Website
        ^                                                          |
        +-------------------- response returns -----------------+
```

The gateway is useful when you want one place to control proxy selection, spread requests over several proxies, avoid unhealthy proxies, limit request rates, and observe what is happening. The included local proxy and website programs let you demonstrate these behaviors without needing a public website or paid proxy service.

2. A simple analogy

Think of the gateway as a dispatch desk:

- **Client devices** are people asking the desk to deliver a package.
- **Requests** are the packages, each addressed to a destination website.
- **Upstream proxies** are delivery routes the desk can use.
- **The scheduler** makes sure work is released at a reasonable rate and one destination does not take over the whole line.
- **The proxy manager** checks which routes are working and chooses one for each job.
- **The upstream engine** carries the package along the chosen route and brings the response back.
- **Metrics** are the desk's activity log and status board.

The analogy is only to make the roles easier to picture. In the actual program, the gateway moves network data over TCP sockets and implements HTTP proxy and SOCKS5 protocols.

3. What is the gateway, and what is a proxy?

A **forward proxy** is a server that a client asks to contact another server on its behalf. Here, the client is configured to use this project as its proxy. The project then uses one of its configured **upstream proxies** to reach the destination.

That creates two separate connections for a normal HTTP request:

The client connects to this gateway.

The gateway connects to the selected upstream proxy.

The upstream proxy connects onward to the destination website.

For the local demo, the path usually looks like:

```
curl -> gateway on port 8000 -> local proxy node on port 8101/8102/8103 -> local target on port 9000
```

The local `proxy_node` and `target_server` are helper programs. They make the project easy to test: a proxy node can be configured to respond slowly or fail some requests, and the target server returns predictable responses. They are not part of the required path when using other upstream proxies and destinations.

4. What happens to a request?

Here is the full journey for an ordinary HTTP request:

- **The client connects.** A command-line tool, browser, or another program connects to the gateway's listening address and sends an HTTP proxy request.
- **The gateway receives bytes.** Its client-facing event loop watches sockets and reads data when it is ready. It can serve many client connections without assigning a separate thread to each one.
- **The request is checked.** The HTTP parser checks the request line, headers, URL, and body format. Size limits and timeouts help reject malformed or excessively large or slow requests.
- **The request is classified.** The destination host is used as a scheduling key. Requests for the same destination share that destination's queue and rate limit.
- **The scheduler decides when it is ready.** It keeps requests in FIFO order per destination, applies that destination's token-bucket limit, enforces the overall queue bound, and expires requests that waited too long.
- **The dispatcher checks capacity.** The gateway only starts as many requests as its current in-flight limit allows. The AIMD controller adjusts that limit based on recent success and failure results.
- **A proxy is selected.** The proxy manager considers configured routing policy, health, circuit-breaker state, measured latency, and active load.
- **The upstream engine forwards the request.** It connects to the chosen proxy, speaks HTTP proxy or SOCKS5 as configured, sends the request, and waits for the response. If appropriate, it may retry using another proxy.
- **The reply returns to the client.** The engine writes the response back. It can reuse eligible keep-alive connections to upstream proxies for later work.
- **The result is recorded.** Request totals, success/failure, latency, and other runtime values are updated. When configured, per-request details are written to CSV and JSONL files.

In short:

```
receive -> parse -> queue/rate-limit -> check capacity -> choose proxy
        -> forward -> receive reply -> return reply -> update metrics
```

5. How HTTPS works here

For an HTTPS website, the client first sends a `CONNECT host:port` request to the gateway. This asks the gateway to make a TCP tunnel to that host and port through an upstream proxy. Once the tunnel is ready, the client and website perform their normal TLS handshake through it.

The gateway relays the encrypted bytes in both directions. It does **not** decrypt them. This protects the normal end-to-end TLS connection, but it also means the gateway cannot see the HTTPS page path, headers, or response body. It can still count tunnel connections and observe tunnel-level information such as duration or the requested host and port, subject to what is logged and measured by the implementation.

6. How the project handles many connections

The gateway uses nonblocking sockets with `epoll`, a Linux mechanism that tells a program which sockets are ready for reading or writing. Instead of waiting on one network operation at a time, a small number of I/O threads revisit connections when they become ready. This lets the gateway make progress on many requests at once without creating a thread for every request.

For CONNECT tunnels, the gateway relays data in both directions. If one side is slow, back-pressure pauses reading from that side until the other side catches up. This keeps temporary buffers bounded. Idle and incomplete connections are also subject to configured time limits.

The `workers.min`, `workers.initial`, and `workers.max` settings are **in-flight request limits**, not the number of operating-system threads. `workers.io_threads` controls the number of I/O loops. The dispatcher and proxy health checker have their own background threads.

7. Main code modules

Gateway startup and request handling

File	In plain language
`src/main.cpp`	Starts the program. It reads command-line settings, loads and checks configuration, reports security warnings, starts the gateway, and shuts it down cleanly when asked. It can also generate a random admin token.
`include/gateway_server.hpp`	The coordinator for the live gateway. It opens the listening socket, accepts clients, reads and parses their requests, sends work to the scheduler, handles local admin requests, and joins the pieces together when responses finish.
`include/types.hpp`	Defines the shared objects passed between modules: settings, a client connection, a request/job, a proxy's current state, and a reusable socket connection. It is the common vocabulary of the program.

Configuration and data parsing

File	In plain language
`include/config.hpp`	Reads the JSON settings file and command-line options. It checks that values are present, have the correct type and range, and loads configured secrets. `--check-config` asks it to validate settings without running the service.
`include/json.hpp`	Reads JSON text and turns it into values that the configuration code can inspect. This is a small parser included in the project.
`include/http.hpp`	Understands HTTP request lines, headers, URLs, body framing, and chunked bodies. It also provides helpers for building responses and safely escaping strings in logs and metrics.
`include/net.hpp`	Provides lower-level networking helpers used across the program: address resolution, opening/listening/connecting sockets, reading HTTP responses, and the SOCKS5 protocol handshake.

Queueing and upstream proxy management

File	In plain language
`include/scheduler.hpp`	Holds requests until they are allowed to run. It keeps a separate FIFO line for each destination, applies token-bucket request limits, shares dispatch time fairly between active destinations, and removes expired work.
`include/proxy_manager.hpp`	Keeps the list of upstream proxies and their current health information. It probes proxies, tracks response time and failures, temporarily avoids proxies that keep failing, and chooses a proxy using the selected policy.
`include/upstream_engine.hpp`	Does the network work after a proxy is chosen. It connects, sends and reads data without blocking an entire thread, manages reusable upstream connections, handles timeouts and allowed retries, and relays CONNECT tunnels.
`include/aimd.hpp`	Changes the allowed number of simultaneous in-flight requests. If a recent group of requests is healthy, it increases the limit gradually; if too many fail or time out, it reduces the limit.

Monitoring and safety

File	In plain language
`include/metrics.hpp`	Counts requests, failures, timeouts, tunnels, active connections, queue size, and other runtime details. It calculates latency percentiles, serves Prometheus-style metrics, and writes configured CSV/JSONL records.
`include/security.hpp`	Checks network addresses and identifies risky configurations, such as exposing an unauthenticated proxy or sending credentials across an untrusted network. It prints warnings; it does not add TLS to the gateway.
`include/fd_budget.hpp`	Makes sure configured connection and idle-pool limits fit within the operating system's file-descriptor allowance. A socket uses a file descriptor, so this is one of the resources that limits connection count.

Dashboard and demo helpers

File	In plain language
`app/gateway_ui.py`	Runs the local dashboard's control service. It can start, stop, and monitor the gateway process, validate and save settings, and provide data to the browser page.
`app/index.html`	The dashboard page you see in the browser. It presents gateway and proxy status, metrics, controls, settings, and log/request information.
`src/proxy_node.cpp`	A pretend upstream proxy for local testing. It supports HTTP proxy requests, CONNECT, and SOCKS5, and can be told to add delay or fail some requests.
`src/target_server.cpp`	A small local website used as a predictable destination in tests and demonstrations. It can return known responses without relying on the internet.
`src/loadgen.cpp`	A traffic generator that sends many requests so throughput and latency can be observed under load.

Tests and build setup

File	In plain language
`tests/unit_tests.cpp`	Checks individual pieces such as parsing and configuration behavior.
`tests/fuzz_parsers.cpp`	Feeds many altered inputs to parsers to look for crashes or incorrect handling. CTest runs a quick sample.
`scripts/e2e_test.py`	Starts local components and tests complete request journeys through the gateway, as a client would experience them.
`scripts/fuzz_network.py`	Sends unusual or malformed network inputs to check how the programs handle hostile peers and unexpected traffic.
`CMakeLists.txt`	Tells CMake how to build the gateway, test utilities, and test suites.

8. Proxy choice, health, and recovery

There can be several upstream proxies in the configuration. A routing policy tells the gateway how to choose among the currently eligible ones:

- **Random:** choose one at random.
- **Round robin:** take turns through the available proxies.
- **Least connections:** prefer the proxy handling the fewest active connections.
- **Lowest latency:** prefer the proxy with the lowest measured response time.
- **Weighted:** score proxies using configured importance for latency, active load, and failure rate.

The gateway periodically probes proxies and updates their health information. After repeated failures, a proxy's **circuit breaker** opens and ordinary work avoids it for a cooldown period. Later, the gateway allows a recovery check (the half-open state). A successful recovery puts the proxy back into service; another failure keeps it out temporarily. This is similar to temporarily taking a broken route off the dispatch board and checking it again later.

Retries are bounded by configuration. Retrying a request is safest before its body has been sent upstream; the gateway uses safeguards so that retrying a non-idempotent request does not casually repeat an operation that may already have reached the destination.

9. Rate limits and concurrency are different

These controls solve different problems:

- A **rate limit** controls how quickly new requests for a destination are released over time. The token bucket allows a configured steady rate plus a short burst.
- A **queue limit and queue timeout** control how much waiting work is retained and how long it can wait.
- The **AIMD concurrency limit** controls how many requests can be in progress at the same time across the gateway.
- The **I/O thread count** controls how many event loops watch sockets. It is not the request concurrency limit.

For example, a gateway might allow many requests to be in flight overall while still releasing requests to one particular domain at a much lower rate.

10. Dashboard, admin endpoints, and metrics

The desktop dashboard is a local control and observation tool. The Python process manages the gateway executable and serves a local web interface; the HTML page displays status and controls. The dashboard does not itself forward the website traffic—the C++ gateway does that.

The gateway also has admin endpoints such as `/health`, `/proxies`, and `/metrics`. They provide health, proxy state, or metrics. Configuration can bind admin endpoints to a separate listener and require a bearer token. Request detail files can be written as CSV or JSON Lines (one JSON object per line), while `/metrics` exposes aggregate values suitable for monitoring tools.

11. Programs you can run

The repository builds several separate programs:

- `gateway` is the main forward-proxy service.
- `proxy_node` imitates upstream proxy servers for controlled experiments.
- `target_server` imitates a website/origin for repeatable local requests.
- `loadgen` generates concurrent client traffic.
- `unit_tests` and `fuzz_parsers` check code behavior.

Docker Compose is an optional way to start a packaged local demo. It is not needed when starting the executables yourself or when configuring the gateway to use reachable upstream proxies.

12. Build and run the local demo

Build the programs and run the registered tests from the repository root:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j"$(nproc)"
ctest --test-dir build --output-on-failure
```

The root README has example commands for starting `target_server`, several `proxy_node` instances, and the gateway, plus configuration details. For a simple request through the local demo, curl can be pointed at the gateway with:

```
curl --noproxy '' -x http://127.0.0.1:8000 http://127.0.0.1:9000/products/123
```

The `--noproxy ''` part tells curl not to bypass the configured proxy for loopback addresses. Without it, an environment `NO_PROXY` setting may cause the command to contact the target directly and skip the gateway.

13. Important security and scope notes

By default, the proxy listens on loopback (`127.0.0.1`) so only programs on the same computer can connect. Changing the listener to a network-facing address allows other devices to reach it, but this must be done deliberately: a forward proxy without client authentication can be abused as an open proxy. Admin tokens and proxy credentials also need protection. The gateway does not provide TLS on its own listeners, so remote access should be limited to a trusted private network or protected by a VPN/tunnel or a TLS front end.

The gateway forwards traffic; it does not make an upstream proxy trustworthy, grant permission to send high traffic to a website, or guarantee any particular speed. Use only destinations and upstream proxies you are authorized to use. HTTPS is tunneled end to end, so the gateway routes the connection but does not inspect its encrypted content.